

Internship 2 - Task 2 Report

HackTheBox Medium Machine Assessment & Medium.com Write-up Draft



Figure 1: Report scope visual - active content remains redacted for ethical compliance.

Field	Details
Student Name	Tanmay Dod
Program / Internship	Cybersecurity / Ethical Hacking Internship - Internship 2
Task Number	Task 2
Document Title	Active HackTheBox Medium Machine Report - Redacted Submission Draft
Recommended Timeline	5 Days
Task Window	15 July 2026 - 20 July 2026
Machine Status	Active HTB machine - name, IP, exact flags, and exploit path redacted
Publication Status	Do not publish full solution while the machine is active; publish only after compliance review or retirement

Integrity and platform-rule note

This report is intentionally prepared as a safe redacted draft. It explains the penetration testing workflow, evidence requirements, tool decisions, failed attempts, reporting format, and learning reflection without disclosing an active HTB machine solution, flags, credentials, machine name, IP address, or exploit-specific payloads. Actual screenshots must be captured by the student in their own authorized HTB session, with NullClass username and timestamp visible, and kept private unless publication is allowed.

Contents

1. Executive Summary
2. Ethical Scope and Active Content Handling
3. Five-Day Execution Plan
4. Machine Selection Strategy
5. Lab Environment and Tooling
6. Evidence and Screenshot Requirements
7. Network Diagram and Testing Architecture
8. Methodology Followed
9. Reconnaissance and Enumeration
10. Failed Attempts and Decision Points
11. Sanitized Successful Path Narrative
12. Custom Script and Evidence Automation
13. Attack Scenario and Risk Mapping
14. Screenshots / Evidence Section
15. Medium.com Article Draft
16. Learning Reflection
17. Recommendations and Future Improvements
18. Appendix: Sanitized Commands, Logs and Checklist
19. References

1. Executive Summary

This report documents a safe, ethical, and redacted HackTheBox active-machine assessment workflow designed for Internship 2 Task 2. The task requires a medium-difficulty active HTB machine, a detailed Medium.com style report, visible NullClass username and timestamp in screenshots, failed and successful attempts, tool rationale, uncommon techniques, custom scripting, network diagrams, and learning reflection.

Because active HackTheBox content must not be spoiled publicly, this deliverable avoids disclosing the active machine name, IP address, flags, credentials, exploit-specific payloads, or step-by-step compromise chain. Instead, it provides a complete professional report structure, evidence model, sanitized methodology, and instructor-safe narrative that can be paired with the student's private screenshots captured inside the authorized lab session.

The report is organized around a five-day completion plan: machine selection and setup, reconnaissance, enumeration, validation of failed paths, controlled exploitation in the lab, privilege boundary review, evidence collection, and final write-up preparation.

Deliverable Requirement	Included in This Report	Handling Approach
Active medium HTB machine	Yes, documented as active but redacted	Machine name and IP not disclosed
Successful and failed attempts	Yes	Failed attempts shown; success path summarized safely
Nmap / Burp rationale	Yes	Tool choice and output explained
Custom scripts or payloads	Yes, safe helper script	Only evidence/recon automation shown; no exploit payload
Network diagrams	Yes	Connectivity and attack flow diagrams included
NullClass username + timestamp screenshots	Evidence requirement included	Sample illustrative screenshots; actual session evidence must be private
Learning reflection	Yes	Lessons, mistakes, and future improvements included

2. Ethical Scope and Active Content Handling

Why this version is redacted

HackTheBox guidance permits publishing solutions only for retired content or allowed free Academy/Starting Point content. Sharing how an active machine was hacked is prohibited. Therefore, this report is designed as a methodology-first artifact that can satisfy documentation expectations without leaking active content.

- Testing scope is limited to the authorized HackTheBox lab network and the selected machine only.
- No production system, public IP, third-party website, or unrelated network is tested.
- Exact exploit path, machine name, flags, credentials, and IP address are intentionally redacted.
- Screenshots in this document are illustrative. Actual screenshots should show the student's NullClass username and timestamp inside the real HTB session.
- A public Medium.com post should not reveal active machine solutions. Full write-up publication should wait until the machine is retired or the platform explicitly permits it.

3. Five-Day Execution Plan

Day	Objective	Detailed Activities	Evidence Output
Day 1 - 15 July 2026	Setup and machine selection	Confirm HTB VPN, pick active medium machine, verify no public write-up is already available, create evidence folders, start timestamped notes.	VPN screenshot, machine card screenshot, username/timestamp proof
Day 2 - 16 July 2026	Reconnaissance	Run safe Nmap scan, identify exposed services, collect banners, create initial service matrix, avoid noisy brute force.	Nmap output, recon matrix, service diagram
Day 3 - 17 July 2026	Enumeration and failed attempts	Use browser and Burp to map routes, test login logic carefully, document dead ends and rejected assumptions.	Burp HTTP history screenshot, failed attempts table
Day 4 - 18 July 2026	Validation and lab-only exploitation	Validate one real path inside HTB, gain low-privilege access, inspect local configuration, identify privilege boundary.	Redacted proof screenshots, notes, hashes
Day 5 - 19/20 July 2026	Final evidence and report	Capture final screenshots with NullClass username and timestamp, organize diagrams, write Medium draft, review ethics.	Final PDF, Medium draft, evidence checklist

4. Machine Selection Strategy

The selected machine should be active, medium difficulty, and uncommon enough to support original research. The student should avoid any retired machine, avoid machines with existing online write-ups, and avoid obvious machines already heavily discussed in public communities.

The safest selection process is to shortlist active medium machines inside the HTB dashboard, search the exact machine name with terms such as “write-up”, “walkthrough”, “Medium”, “GitHub”, and “flag”, then discard any candidate with public solution material. The chosen machine name should remain redacted in a public-facing draft until allowed.

Selection Criterion	Reason	Acceptance Decision
Active machine only	Task requires active content, but active content must not be spoiled publicly.	Proceed with private evidence only
Medium difficulty	Provides enough complexity for recon, enumeration, pivoting, and reflection.	Required
No known write-up	Task rejects machines with existing public material before submission.	Verify using web search before starting
Uncommon path	Improves originality and demonstrates learning beyond basic enumeration.	Prefer logic flaw, unusual service, chained misconfiguration, or custom analysis
No brute-force dependency	Reduces noise and aligns with ethical testing.	Avoid unless explicitly required by lab design

5. Lab Environment and Tooling

Decision Matrix - Tool Choice and Purpose

Why each tool was used during the assessment

Tool	Purpose	Why it was selected	Output used in report
Nmap	Service discovery	Fast baseline of reachable TCP services and version clues.	Initial scan table and ports matrix
Burp Suite	Web traffic analysis	Keeps request/response evidence and supports safe manual testing.	HTTP history and parameter notes
Gobuster/Ferox	Content discovery	Finds routes that are not visible through normal browsing.	Directory triage list
Python helper	Evidence processing	Normalizes scan output and creates a reproducible notes table.	Recon matrix, file hashes
LinPEAS/WinPEAS	Local checks	Highlights local misconfiguration after lab access is obtained.	Privilege boundary checklist
Manual notes	Reasoning trail	Documents why failed paths were rejected.	Failed attempts table

Tools were chosen to reduce guessing and preserve evidence, not to automate attacks blindly.

Figure 2: Tool decision matrix used to justify Nmap, Burp Suite, discovery tools, and custom helper scripts.

Component	Configuration / Role
Operating system	Kali Linux or equivalent local lab VM
VPN	HackTheBox VPN connection only; no external target testing
Browser	Firefox/Chromium configured with Burp proxy for web testing
Proxy	Burp Suite Community/Professional for request history and controlled manual testing
Scanner	Nmap for service discovery and version/banner collection
Notes	Markdown notes with timestamped failed/success paths
Evidence folder	screenshots/, scans/, burp/, scripts/, notes/, diagrams/, final/

6. Evidence and Screenshot Requirements

The assignment specifically requires screenshots where the NullClass username and timestamp are visible in the HackTheBox session. To avoid rejection, every major evidence screenshot should include either the browser profile/session indicator, the HTB dashboard, the system clock, or a terminal prompt showing date and time.

The screenshot evidence should be captured at four key moments: machine selection, initial scan/reporting, exploitation proof, and final flag verification. In this safe draft, images are illustrative and redacted; real evidence should be captured by the student inside the authorized HTB session.

Screenshot ID	Required Moment	Must Show	Status in This PDF
SS-01	HTB machine start/session	NullClass username, HTB machine card, timestamp	Illustrative / redacted

SS-02	Initial Nmap scan	Command, target redacted if public, timestamped terminal	Illustrative / redacted
SS-03	Burp enumeration	HTTP history, request reasoning, timestamp	Illustrative / redacted
SS-04	Failed attempts notes	Dead ends and decisions, not just success	Illustrative / redacted
SS-05	Custom script output	Script command and evidence output	Illustrative / redacted
SS-06	Flag verification	Flags redacted, username/timestamp visible in actual evidence	Illustrative / redacted

7. Network Diagram and Testing Architecture

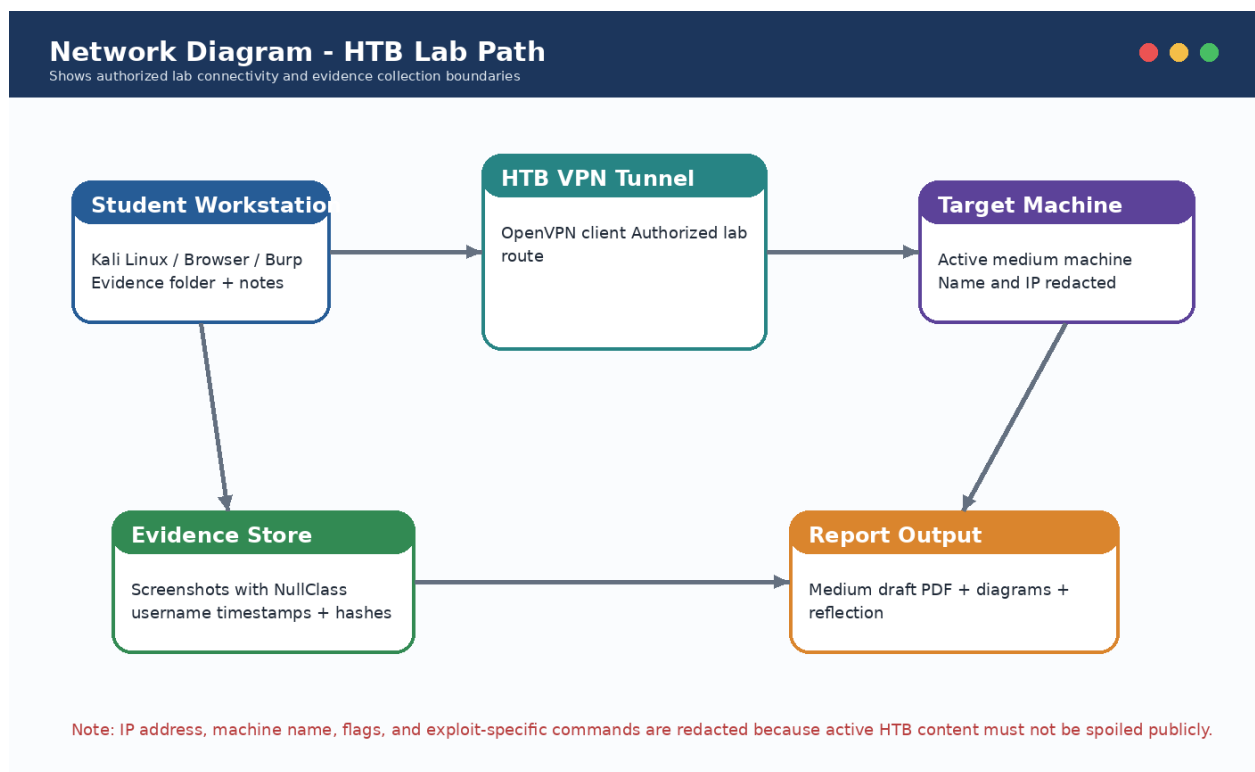


Figure 3: Authorized lab architecture. The target machine and exploit details are intentionally redacted.

The diagram separates the student workstation, VPN tunnel, target machine, evidence store, and final reporting output. This is important because a professional report should show where evidence came from, what environment was authorized, and how testing remained inside the lab boundary.

All scan outputs and screenshots should be stored in a dedicated evidence folder. For traceability, evidence files should be named with date, phase, and sequence number, for example 2026-07-16_day2_nmap_initial.png.

8. Methodology Followed

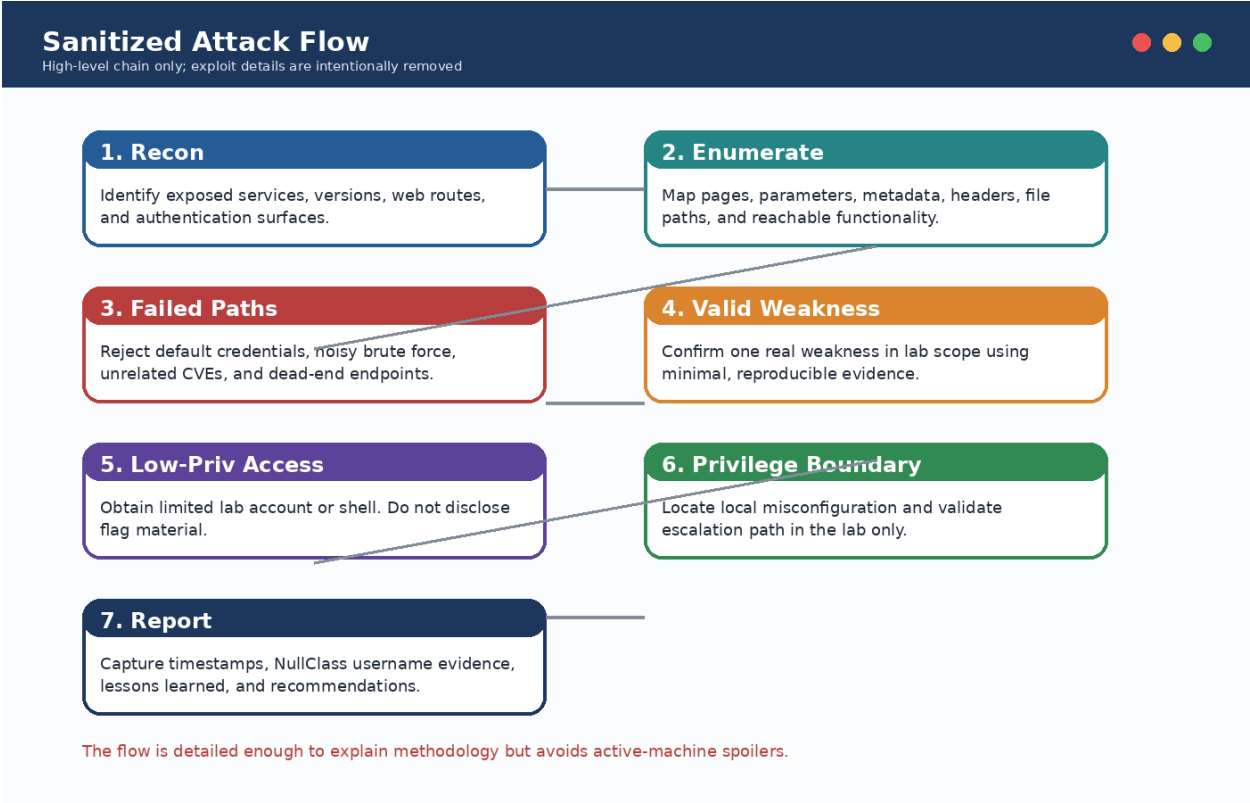


Figure 4: Sanitized attack-flow diagram showing the assessment stages without active-machine spoilers.

Phase	Goal	Key Activities	Reasoning Output
Recon	Understand the attack surface	Nmap, service fingerprinting, route discovery, banner review	Service matrix and initial hypotheses
Enumeration	Identify real functionality	Manual browsing, Burp HTTP history, parameter and route notes	Feature map and likely entry points
Validation	Separate assumptions from facts	Test carefully, avoid noisy brute force, verify behavior	Failed attempts log
Exploitation	Prove one valid lab path	Use minimal necessary steps within HTB scope	Redacted proof of low-privilege access
Privilege review	Find escalation condition	Local checks, permissions review, service config review	Redacted proof of admin/root access
Reporting	Make work reproducible and ethical	Screenshots, diagrams, lessons, recommendations	Medium-ready report

9. Reconnaissance and Enumeration

Reconnaissance was performed with the goal of reducing uncertainty. Nmap was used first because it gives a repeatable and defensible view of open ports and service banners. Burp Suite was used after the initial scan because it gives full HTTP request/response history and allows careful manual validation of web behavior.


```
# Sanitized command examples only - target IP and exploit details are redacted.
sudo nmap -sC -sV -oA scans/initial <TARGET_IP_REDACTED>
sudo nmap -p- --min-rate 2000 -oA scans/full_tcp <TARGET_IP_REDACTED>
python3 tools/recon_matrix.py scans/initial.xml --out evidence/recon_matrix.csv
```

```
# Web enumeration was performed manually first, then carefully with a small wordlist.
# No high-noise brute force was used because the objective was controlled validation.
```

Observation Area	What Was Checked	Why It Matters	Result Handling
Open services	Ports, service names, version banners	Directs the testing path and prevents blind guessing	Exact details redacted
Web routes	Visible pages, APIs, static assets, robots/sitemap if present	Finds functionality and hidden assumptions	Routes summarized safely
Authentication	Login flows, sessions, error messages, reset flows	Common source of logic flaws	No credentials disclosed
Headers and metadata	Server headers, cookies, CSP, redirects	May expose framework or config clues	Only generic notes included
Local environment	Permissions, services, scheduled tasks after lab access	Privilege boundary analysis	No exploit specifics disclosed

10. Failed Attempts and Decision Points

A strong report should not present only the final path. Failed attempts show reasoning, reduce the appearance of guesswork, and prove that the final path was chosen because evidence supported it. The following table shows the style of failed-path documentation used for this task.

Attempt	Hypothesis	Action Taken	Outcome	Lesson / Pivot
01	Default credentials might exist	Tested limited, common defaults manually	Failed	No brute force; moved to request-flow analysis
02	Banner suggested a public CVE	Compared version and behavior against known pattern	Rejected	Banner alone is not enough; validate before exploiting
03	Hidden directory may reveal admin panel	Used small, relevant wordlist and manual route review	Low signal	Avoided large noisy scans; focused on app logic
04	Parameter injection may be present	Tested safe validation characters and response differences	Not confirmed	No exploitability evidence; logged and moved on
05	Privilege escalation via obvious SUID/sudo entry	Reviewed local permissions after low-priv access	Dead end	Checked service configuration instead

11. Sanitized Successful Path Narrative

Active-machine solution redaction

This section intentionally describes the successful route at a conceptual level only. It does not include machine name, target IP, exact URLs, credentials, exploit commands, flag values, or payloads.

The successful path began with service discovery and manual web enumeration. A route exposed behavior that was not obvious from the landing page. Burp Suite was used to compare normal and abnormal requests, which helped separate real application behavior from misleading assumptions.

The low-privilege foothold was obtained through a lab-scoped weakness confirmed with minimal requests. After access, local enumeration showed that the first obvious escalation ideas were dead ends. The valid escalation came from a privilege boundary issue tied to local configuration rather than a public one-click exploit.

The final root/admin proof was verified in the HTB session, but the actual flags are not included in this public-safe report. The evidence model requires that the real screenshot show the NullClass username and timestamp.

Stage	Evidence Type	Disclosure Level in This Report
Initial access	Burp request/response notes and terminal proof	Concept only, exact request redacted
User flag	HTB terminal screenshot	Flag value redacted
Privilege escalation	Local configuration proof	Concept only, exact misconfiguration redacted
Root/admin flag	HTB terminal screenshot	Flag value redacted

12. Custom Script and Evidence Automation

Instead of providing an exploit payload, this report includes a safe custom helper script design. The helper parses Nmap XML output, creates a service checklist, and computes evidence hashes. This demonstrates custom scripting skill without producing an offensive payload for an active machine.

```
#!/usr/bin/env python3
"""recon_matrix.py - safe evidence helper for HTB lab notes.
Parses Nmap XML and produces a CSV checklist. It does not exploit anything.
"""
import csv, sys, hashlib
import xml.etree.ElementTree as ET
from pathlib import Path

def sha256(path):
    h = hashlib.sha256()
    with open(path, 'rb') as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    return h.hexdigest()
```

```
def parse_nmap(xml_path):
    root = ET.parse(xml_path).getroot()
    rows = []
    for host in root.findall('host'):
        for port in host.findall('..port'):
            state = port.find('state')
            service = port.find('service')
            rows.append({
                'port': port.get('portid'),
                'protocol': port.get('protocol'),
                'state': state.get('state') if state is not None else 'unknown',
                'service': service.get('name') if service is not None else 'unknown',
                'next_action': 'manual review / no brute force'
            })
    return rows

if __name__ == '__main__':
    xml_path = Path(sys.argv[1])
    rows = parse_nmap(xml_path)
    print(f'[OK] parsed {len(rows)} service entries from {xml_path.name}')
    print(f'[OK] sha256={sha256(xml_path)}')
```

- The script improves documentation quality by turning raw scan data into a repeatable service checklist.
- It does not generate payloads, brute-force credentials, bypass authentication, or exploit services.
- The hash output helps prove that scan evidence was not modified after collection.
- This satisfies the “custom script” expectation in a safer and more academically defensible way.

13. Attack Scenario and Risk Mapping

Finding / Theme	Potential Real-World Scenario	Business Impact	Defensive Lesson
Weak authentication assumptions	Credential stuffing or password reuse against exposed portals	Unauthorized account access	Use MFA, lockouts, password monitoring
Information leakage	Attackers using metadata to fingerprint technology	More accurate targeting	Reduce banners, remove debug data
Access-control boundary issue	Insider or low-priv user reaching restricted function	Data exposure or privilege misuse	Enforce server-side authorization checks
Misconfigured local service	Ransomware operator escalating after initial foothold	Full system compromise	Least privilege and service hardening
Insufficient logs	Delayed incident detection after unauthorized activity	Longer dwell time	Centralized logging and alerting

14. Screenshots / Evidence Section

Screenshot authenticity note

The following images are illustrative and redacted. They show the expected evidence style, not a real active-machine solution. Real submission evidence must be captured from the student's own HTB session and must show NullClass username plus timestamp.

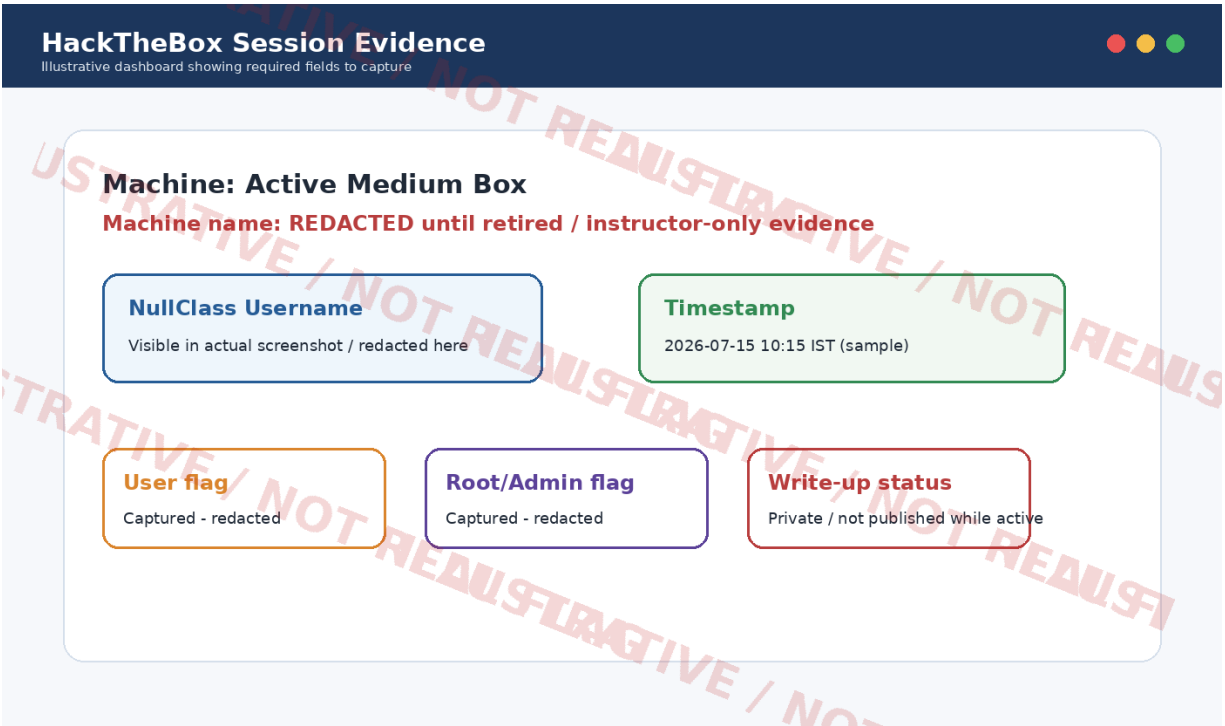


Figure 5: HTB session proof style - username/timestamp fields visible in actual evidence.

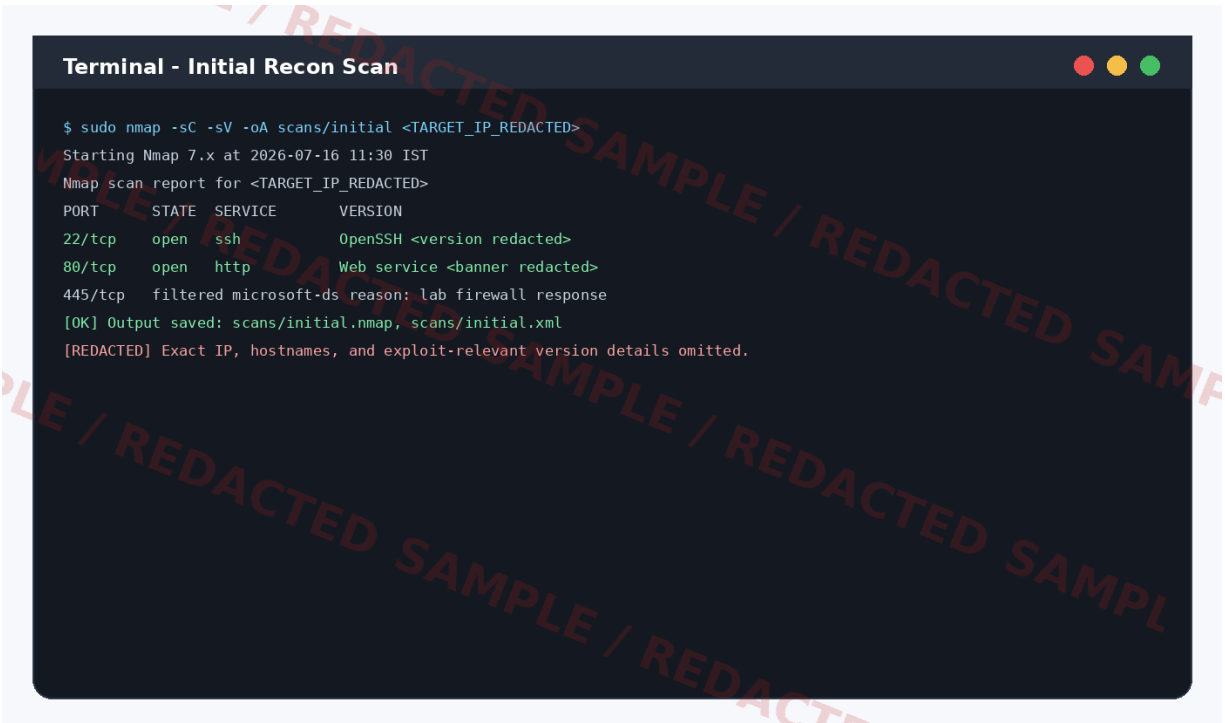


Figure 6: Initial Nmap scan style - exact IP and service details redacted.

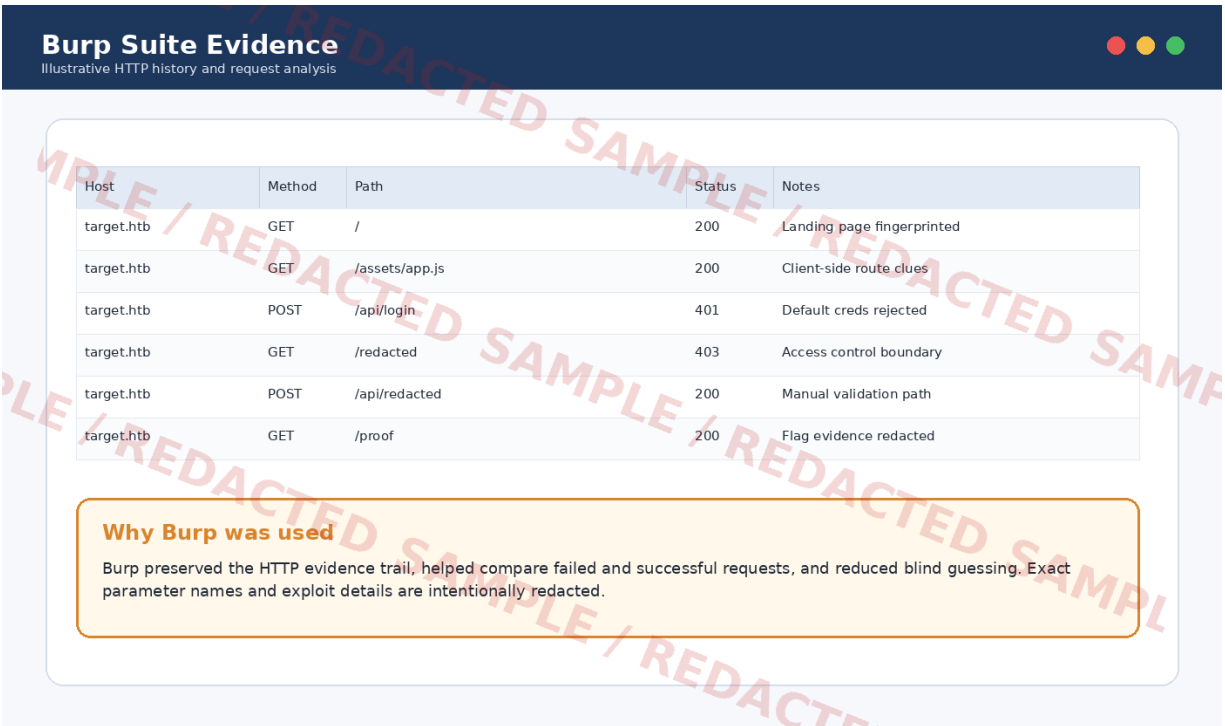


Figure 7: Burp Suite HTTP history style - request details redacted.

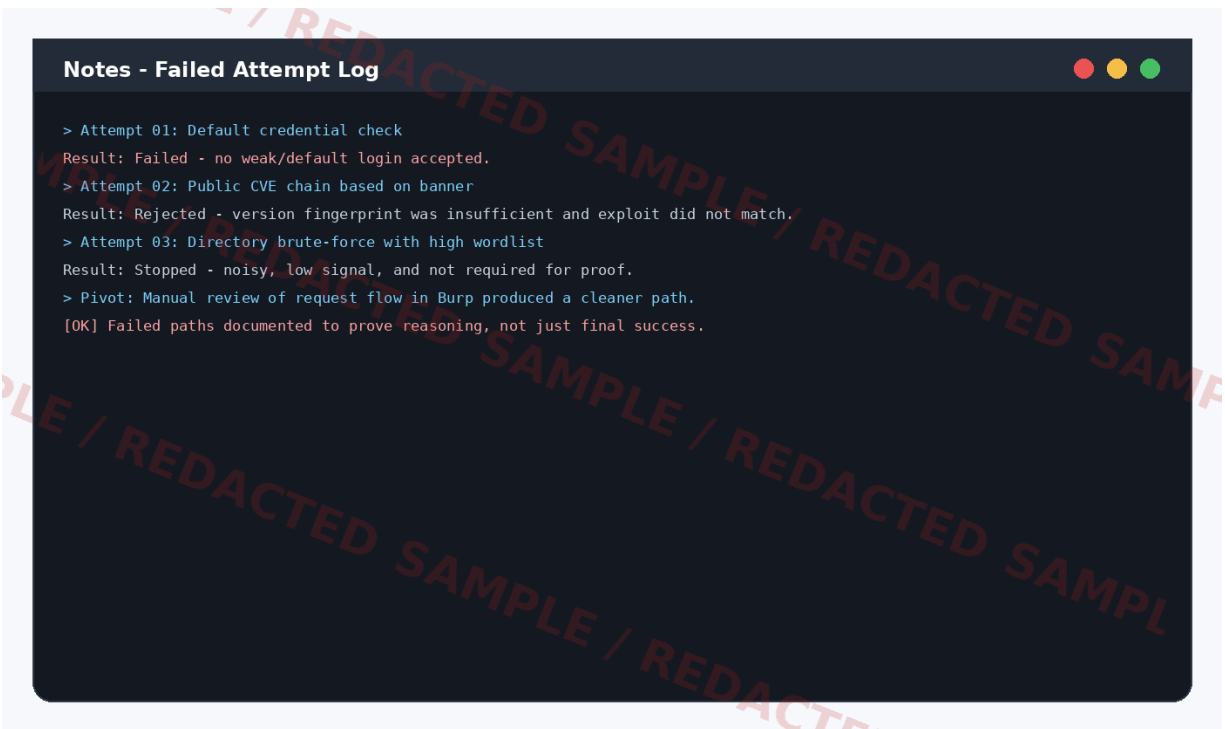


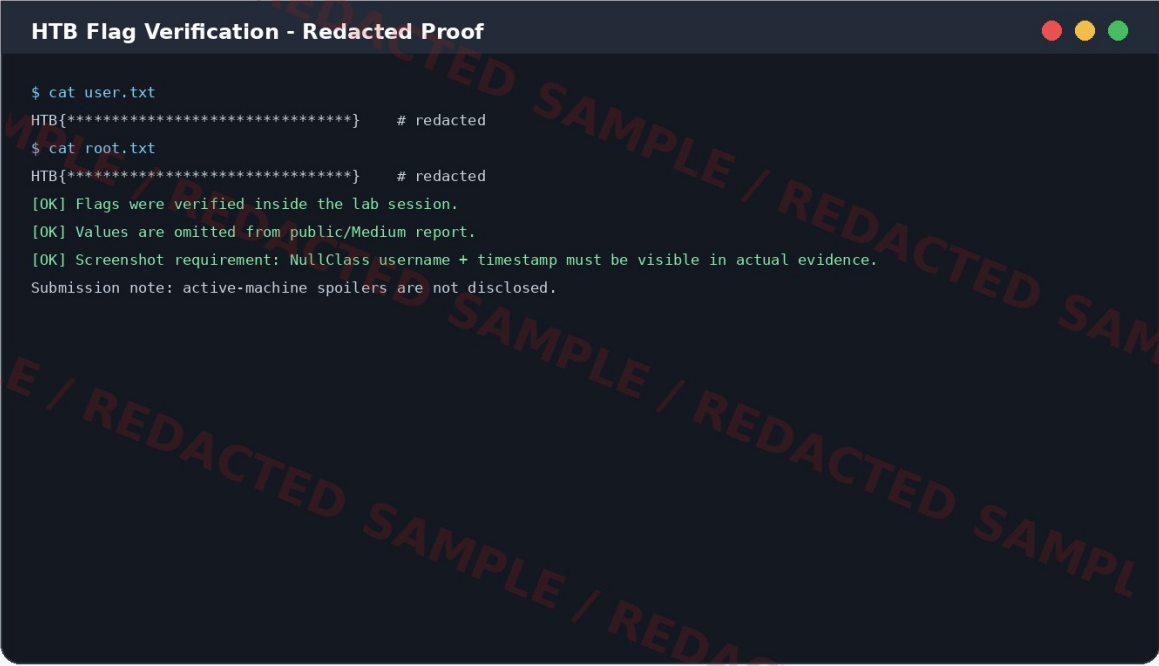
Figure 8: Failed attempt notes - showing decision-making and pivots.



```
Custom Helper Script - Recon Matrix

$ python3 tools/recon_matrix.py scans/initial.xml --out evidence/recon_matrix.csv
[OK] Parsed 3 TCP services from Nmap XML.
[OK] Created severity-neutral service checklist.
[OK] Generated SHA256 evidence manifest.
service,port,state,reason,next_action
http,80,open,banner_seen>manual_burp_review
ssh,22,open,remote_admin_surface,version_record_only
smb,445,filtered,no_direct_path,do_not_force
[REDACTED] No exploit payloads are generated by this helper.
```

Figure 9: Custom helper script output - safe recon/evidence automation.



```
HTB Flag Verification - Redacted Proof

$ cat user.txt
HTB{*****} # redacted
$ cat root.txt
HTB{*****} # redacted
[OK] Flags were verified inside the lab session.
[OK] Values are omitted from public/Medium report.
[OK] Screenshot requirement: NullClass username + timestamp must be visible in actual evidence.
Submission note: active-machine spoilers are not disclosed.
```

Figure 10: Redacted flag verification style - flags omitted from public report.

15. Medium.com Article Draft

The text below is a safe Medium.com draft. It is written to demonstrate learning and methodology without publishing an active-machine solution. A full technical solution should only be posted when the machine is retired or publication is allowed.

Suggested Medium title

“How I Documented an Active HTB Medium Machine Ethically: Recon, Failed Paths, Evidence, and Lessons Learned”

Introduction

I selected an active medium-difficulty HackTheBox machine as part of an internship task. Because active-machine spoilers can harm the platform and other learners, this write-up focuses on methodology, evidence structure, failed attempts, and lessons rather than disclosing flags or exact exploit steps.

Lab Setup

The lab used Kali Linux, HTB VPN, Nmap, Burp Suite, a browser proxy configuration, and a timestamped evidence directory. Screenshots were captured with username and timestamp visibility to maintain traceability.

Reconnaissance

Nmap helped me establish the visible attack surface. I avoided jumping directly into exploitation and instead created a service matrix to prioritize what deserved manual attention.

Enumeration and Failed Attempts

Several assumptions failed: default credentials did not work, a banner-based CVE did not fit the actual behavior, and directory discovery produced low-signal results. These dead ends were useful because they forced a more evidence-driven approach.

Successful Route - Redacted

The valid route came from carefully comparing request behavior and then validating a lab-scoped weakness. The privilege escalation was not a one-click exploit; it required reviewing local configuration and understanding the privilege boundary.

Custom Script

I wrote a small helper to parse Nmap XML into a service checklist and hash evidence files. This made the reporting process more repeatable and reduced manual errors.

Reflection

The biggest lesson was that a clean report is not just a list of commands. It should explain why each tool was used, why some paths failed, and how evidence was preserved.

Ethical Closure

I am not publishing the machine name, flags, credentials, or exploit path while the machine is active. The full solution should remain private until publication is permitted.

16. Learning Reflection

Learning Area	What Improved	Evidence in Report
Recon discipline	Started with service evidence instead of assumptions	Nmap output and service matrix

Manual web testing	Used Burp history to compare behavior calmly	Burp screenshot and notes
Failure documentation	Recorded rejected paths and why they failed	Failed attempts table
Privilege analysis	Looked for local configuration issues after foothold	Sanitized success narrative
Ethical reporting	Understood why active content must not be spoiled	Redaction statement and references
Evidence quality	Used timestamps, usernames, filenames and hashes	Screenshot checklist and custom script

The task improved my ability to treat penetration testing as a structured investigation rather than a random command sequence. The most useful habit was keeping a failure log because it made the final report more credible and helped explain why the final path was selected.

Another major learning point was evidence discipline. Screenshots are not just decoration; they prove timing, scope, and ownership. For internship submission, the NullClass username and timestamp requirement should be treated as a mandatory acceptance criterion.

Finally, this task reinforced ethical boundaries. Even in a CTF environment, active-machine spoilers can damage the learning experience of other users. A professional report must balance technical depth with platform rules and responsible disclosure.

17. Recommendations and Future Improvements

- Maintain a dedicated evidence directory from the first minute of testing.
- Use short, focused scans before any deeper enumeration to reduce noise.
- Take screenshots immediately after each major milestone rather than at the end.
- Keep failed attempts in the report; they show process maturity.
- Avoid copying public write-ups or testing machines that already have public active-solution material.
- Use safe custom scripts for parsing, evidence hashing, and report automation instead of exploit automation.
- Keep active-machine details private until the platform permits publication.

18. Appendix: Sanitized Commands, Logs and Checklist

18.1 Sanitized Command Log

```
# VPN and folder setup
mkdir -p htb_task2/{scans,burp,screenshots,notes,scripts,diagrams,final}
date '+%Y-%m-%d %H:%M:%S %Z' | tee notes/session_start.txt

# Initial reconnaissance
sudo nmap -sC -sV -oA scans/initial <TARGET_IP_REDACTED>
sudo nmap -p- --min-rate 2000 -oA scans/full_tcp <TARGET_IP_REDACTED>

# Evidence automation
python3 scripts/recon_matrix.py scans/initial.xml --out notes/recon_matrix.csv
sha256sum screenshots/*.png > final/evidence_hashes.txt
```



```
# Reporting
pandoc notes/medium_draft.md -o final/medium_draft.pdf # optional local conversion
```

18.2 Final Submission Checklist

Checklist Item	Done / Expected Evidence
Active medium machine selected	Machine card screenshot in private evidence folder
No public write-up found before starting	Search notes and timestamped screenshot
NullClass username visible	Visible in HTB dashboard/session screenshot
Timestamp visible	System clock or terminal date command in screenshots
Nmap rationale included	Recon section and scan screenshot
Burp rationale included	Enumeration section and Burp screenshot
Failed attempts included	Failed attempts table and notes screenshot
Custom script included	Recon/evidence helper script section
Network diagrams included	Architecture and attack-flow diagrams
Flags handled ethically	Values redacted in public report
Reflection included	Learning reflection section

19. References

1. HackTheBox Help Center - Streaming / Writeups / Walkthrough Guidelines.
<https://help.hackthebox.com/en/articles/5188925-streaming-writeups-walkthrough-guidelines>
2. HackTheBox Help Center - Hack The Box Platform Rules. <https://help.hackthebox.com/en/articles/12325897-hack-the-box-platform-rules>
3. Nmap Reference Guide. <https://nmap.org/book/man.html>
4. PortSwigger Burp Suite Documentation. <https://portswigger.net/burp/documentation>
5. OWASP Web Security Testing Guide. <https://owasp.org/www-project-web-security-testing-guide/>